

A INTERNSHIP PROJECT

ON

“Developing a Python-Based Project College Management System”

A Summer Internship Project carried out of six phrase, submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

BY

B.Gayathri Bhargavi

Regd.No: 229X1A3204



Department of Computer Science and Engineering

G. Pulla Reddy Engineering College (Autonomous):

Kurnool – 518007, Andhra Pradesh, India

(Affiliated to Jawaharlal Nehru Technological University – Anantapur)

Summer Internship Project carried out at : Six Phrase, Coimbatore

Summer Internship Project Report

**Submitted in accordance with the requirement for the degree
of BACHELOR OF TECHNOLOGY**

Name of the College: G. PULLA REDDY ENGINEERING COLLEGE

Department: Computer Science and Engineering in Data Science (CSD)

Name of the Mentor: PRAVEEN KUMAR PARADESI

Company Name: Six Phrase, Coimbatore

Title of the Project: College Management System

Name of the Student: B. Gayathri Bhargavi

Regd Number: 229X1A3204

Program of Study: Summer Internship

Date of Submission:

Table of Contents

1.ABSTRACT	4
2.INTRODUCTION.....	5
3.IMPLEMENTATION	6
4.RESULTS.....	32
5.FUTURE WORKS	36
6.CONCLUSION	37
7.REFERENCES	38
8. GITHUB REPOSITORY	38

ABSTRACT

The College Management System (CMS) is a desktop-based application developed using Python and Tkinter to manage and streamline academic records for both students and faculty. This system provides functionality for login authentication, separate dashboards for students and faculty, GPA calculation, Excel data integration, and performance visualization through charts. It addresses the common challenges of traditional manual record-keeping, such as inefficiency, redundancy, and error-proneness. The primary objective of the system is to digitalize the entire process of storing student academic information and make it accessible and modifiable only by authenticated users. It ensures better data security, reduces human error, and allows ease of access. The system further provides performance insights using visual GPA trends and data filtering, allowing faculty to make informed academic decisions. This project combines user interface development with backend data management, offering a compact and effective solution for academic institutions.

INTRODUCTION

Statement of Project:

This project involves the development of a Python-based College Management System using Tkinter. It allows faculty to manage student records, calculate GPA, and analyze performance, while students can securely access their academic details. The system integrates Excel data handling and visual analytics to enhance academic administration.

Brief of Project:

In many educational institutions, student data is managed manually or using scattered digital tools like spreadsheets, which can be inefficient, insecure, and difficult to maintain. The purpose of this project is to develop a centralized College Management System using Python and its Tkinter GUI toolkit. The system is designed to be user-friendly and role-specific, offering custom dashboards for students and faculty. Students can view their personal details and academic performance, while faculty members can log in to upload grades, calculate GPA, and view student performance analytics. The software offers enhanced usability through its graphical interface and streamlined navigation. With the growing need for digital transformation in education, this system aims to reduce administrative burden, avoid data duplication, and facilitate quicker access to academic records. Additionally, by integrating chart visualizations and Excel file handling, the system provides scalable and future-ready functionalities.

IMPLEMENTATION

The project was implemented using **Python** as the core programming language and **Tkinter** for the GUI. The login module supports two types of users: **faculty** and **students**. Authentication is handled using hardcoded or file-based credentials. Upon successful login, the user is directed to a role-specific dashboard. The faculty dashboard enables uploading of student data using Excel files (using pandas and openpyxl), calculation of GPA from marks, filtering data based on semester or branch, and visualizing performance using matplotlib charts. The student dashboard displays academic records, GPA summary, and any faculty notes or comments. Each GUI element such as frames, buttons, labels, and input fields was created using Tkinter widgets, ensuring responsive and intuitive design. The backend handles data using CSV/Excel files and dictionary-based in-memory storage. The modular coding approach allows ease of maintenance and future feature upgrades. Additional Python libraries such as matplotlib, pandas, and tkFileDialog were used for charting, data handling, and file operations respectively.

SOURCE CODE:

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
from PIL import Image, ImageTk
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import seaborn as sns

# ===== DATA LOADING =====

student_data_path = r"C:\Users\admin\Downloads\Updated_Student_Data_With_Fails (1).xlsx"
faculty_data_path = r"C:\Users\admin\Downloads\Faculty_Info final.xlsx"
timetable_data_path = r"D:\PROJECT 6PHRASE\TIMETABLE.xlsx"

student_df = pd.read_excel(student_data_path)
faculty_df = pd.read_excel(faculty_data_path, sheet_name="Sheet1")
faculty_credentials = dict(zip(faculty_df["Username"], faculty_df["Password"]))
student_df['Percentage'] = pd.to_numeric(student_df['Percentage'], errors='coerce')

# ===== GPA CHART =====
def plot_gpa_chart(student_row):
    year = student_row['Year']
```

```

sem_labels = [f"Sem{i}" for i in range(1, year * 2 + 1)]
gpas = []
for i in range(1, year * 2 + 1):
    gpa = student_row.get(f"Sem{i}(GPA)")
    if pd.isna(gpa):
        gpa = student_row.get(f"Sem{i}(CGPA)")
    gpas.append(gpa)
plt.figure(figsize=(5, 3))
plt.plot(sem_labels, gpas, marker='o', linestyle='-', color='#3498db')
plt.title("GPA Trend")
plt.xlabel("Semester")
plt.ylabel("GPA")
plt.ylim(0, 10)
plt.grid(True)
plt.tight_layout()
chart_path = "gpa_chart.png"
plt.savefig(chart_path)
plt.close()
return chart_path

# ===== CLASS INCHARGE APP =====
class ClassInchargeApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Class Incharge Management System")
        self.root.geometry("1200x800")
        self.root.configure(bg="#f0f4f7")
        self.student_df = None
        self.filtered_df = None
        self.student_data_path = student_data_path
        self.create_header()
        self.create_main_interface()
        self.load_data(self.student_data_path)

    def create_header(self):
        header_frame = tk.Frame(self.root, bg="#2c3e50", height=80)
        header_frame.pack(fill=tk.X)
        header_frame.pack_propagate(False)
        title_label = tk.Label(
            header_frame,
            text="CLASS INCHARGE MANAGEMENT SYSTEM",
            font=("Segoe UI", 20, "bold"),
            bg="#2c3e50",
            fg="white"
        )
        title_label.pack(expand=True)

    def create_main_interface(self):
        main_container = tk.Frame(self.root, bg="#f0f4f7")
        main_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
        self.create_sidebar(main_container)

```

```

self.create_content_area(main_container)

def create_sidebar(self, parent):
    sidebar = tk.Frame(parent, bg="white", width=300, relief=tk.RAISED, bd=1)
    sidebar.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 10))
    sidebar.pack_propagate(False)
    filter_frame = tk.LabelFrame(sidebar, text=" Filters",
                                font=("Segoe UI", 12, "bold"),
                                bg="white", fg="#2c3e50", padx=10, pady=10)
    filter_frame.pack(fill=tk.X, padx=10, pady=10)
    tk.Label(filter_frame, text="Year:", font=("Segoe UI", 10, "bold"), bg="white").pack(anchor="w", pady=(5,0))
    self.year_var = tk.StringVar(value="All")
    year_combo = ttk.Combobox(filter_frame, textvariable=self.year_var,
                              values=["All", "1", "2", "3", "4"], state="readonly")
    year_combo.pack(fill=tk.X, pady=(0,10))
    tk.Label(filter_frame, text="Branch:", font=("Segoe UI", 10, "bold"), bg="white").pack(anchor="w", pady=(5,0))
    self.branch_var = tk.StringVar(value="All")
    branch_combo = ttk.Combobox(filter_frame, textvariable=self.branch_var,
                                values=["All", "CSE", "ECE", "EEE", "MECH", "CIV"],
                                state="readonly")
    branch_combo.pack(fill=tk.X, pady=(0,10))
    tk.Label(filter_frame, text="Semester:", font=("Segoe UI", 10, "bold"), bg="white").pack(anchor="w", pady=(5,0))
    self.sem_var = tk.StringVar(value="All")
    sem_combo = ttk.Combobox(filter_frame, textvariable=self.sem_var,
                              values=["All", "Sem1", "Sem2", "Sem3", "Sem4", "Sem5", "Sem6", "Sem7", "Sem8"],
                              state="readonly")
    sem_combo.pack(fill=tk.X, pady=(0,10))
    apply_btn = tk.Button(
        filter_frame,
        text=" Apply Filters",
        font=("Segoe UI", 10, "bold"),
        bg="#27ae60",
        fg="white",
        activebackground="#219150",
        command=self.apply_filters,
        relief=tk.FLAT,
        pady=8
    )
    apply_btn.pack(fill=tk.X, pady=10)
    analytics_frame = tk.LabelFrame(sidebar, text=" Quick Analytics",
                                   font=("Segoe UI", 12, "bold"),
                                   bg="white", fg="#2c3e50", padx=10, pady=10)
    analytics_frame.pack(fill=tk.X, padx=10, pady=10)
    analytics_btn = tk.Button(
        analytics_frame,
        text=" View Analytics",
        font=("Segoe UI", 10, "bold"),
        bg="#e74c3c",
        fg="white",
        activebackground="#c0392b",
        command=self.show_analytics,

```



```

        relief=tk.FLAT,
        pady=8
    )
analytics_btn.pack(fill=tk.X, pady=5)
export_btn = tk.Button(
    analytics_frame,
    text="Export Data",
    font=("Segoe UI", 10, "bold"),
    bg="#f39c12",
    fg="white",
    activebackground="#e67e22",
    command=self.export_data,
    relief=tk.FLAT,
    pady=8
)
export_btn.pack(fill=tk.X, pady=5)

def create_content_area(self, parent):
    self.content_frame = tk.Frame(parent, bg="white", relief=tk.RAISED, bd=1)
    self.content_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
    if self.student_df is None:
        tk.Label(self.content_frame,
            text="Loading student data...",
            font=("Segoe UI", 14), bg="white", fg="#7f8c8d").pack(expand=True)

def load_data(self, file_path):
    try:
        self.student_df = pd.read_excel(file_path)
        self.show_data_summary()
    except FileNotFoundError:
        messagebox.showerror("Error", f"File not found: {file_path}\nPlease ensure the file exists at the specified path.")
        for widget in self.content_frame.winfo_children():
            widget.destroy()
        tk.Label(self.content_frame,
            text=f"Error: Data file not found at {file_path}",
            font=("Segoe UI", 14), bg="white", fg="red").pack(expand=True)
    except Exception as e:
        messagebox.showerror("Error", f"Failed to load data: {str(e)}")
        for widget in self.content_frame.winfo_children():
            widget.destroy()
        tk.Label(self.content_frame,
            text=f"Error loading data: {str(e)}",
            font=("Segoe UI", 14), bg="white", fg="red").pack(expand=True)

def show_data_summary(self):
    if self.student_df is None:
        return
    for widget in self.content_frame.winfo_children():
        widget.destroy()
    summary_header = tk.Frame(self.content_frame, bg="#ecf0f1", pady=15)
    summary_header.pack(fill=tk.X, padx=10, pady=(10,0))

```

```

tk.Label(
    summary_header,
    text=" Data Summary",
    font=("Segoe UI", 16, "bold"),
    bg="#ecf0f1",
    fg="#2c3e50"
).pack()
stats_frame = tk.Frame(self.content_frame, bg="white")
stats_frame.pack(fill=tk.X, padx=10, pady=10)
total_students = len(self.student_df)
branches = self.student_df['Branch'].unique() if 'Branch' in self.student_df.columns else []
years = self.student_df['Year'].unique() if 'Year' in self.student_df.columns else []
stats_text = f"""
Total Students: {total_students}
Branches: {' , '.join(map(str, branches))}
Years: {' , '.join(map(str, sorted(years)))}
"""
tk.Label(
    stats_frame,
    text=stats_text,
    font=("Segoe UI", 12),
    bg="white",
    fg="#2c3e50",
    justify=tk.LEFT
).pack(anchor="w", padx=20, pady=10)
self.show_student_table()

def apply_filters(self):
    if self.student_df is None:
        messagebox.showwarning("Warning", "Student data is not loaded. Cannot apply filters.")
        return
    self.filtered_df = self.student_df.copy()
    year = self.year_var.get()
    branch = self.branch_var.get()
    semester = self.sem_var.get()
    if year != "All":
        if 'Year' in self.filtered_df.columns:
            self.filtered_df = self.filtered_df[self.filtered_df["Year"].astype(str) == year]
    if branch != "All":
        if 'Branch' in self.filtered_df.columns:
            self.filtered_df = self.filtered_df[self.filtered_df["Branch"] == branch]
    if semester != "All":
        if "Semester" in self.filtered_df.columns:
            self.filtered_df = self.filtered_df[self.filtered_df["Semester"] == semester]
    self.show_student_table()

def show_student_table(self):
    data_to_show = self.filtered_df if self.filtered_df is not None else self.student_df
    if data_to_show is None or data_to_show.empty:
        for widget in self.content_frame.winfo_children():
            widget.destroy()

```

```

no_data_label = tk.Label(self.content_frame, text="No student data to display or filters resulted in empty data.",
                          font=("Segoe UI", 12), bg="white", fg="#7f8c8d")
no_data_label.pack(expand=True)
return
for widget in self.content_frame.winfo_children():
    widget.destroy()
table_frame = tk.Frame(self.content_frame, bg="white")
table_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
header_frame = tk.Frame(table_frame, bg="#34495e", pady=10)
header_frame.pack(fill=tk.X)
tk.Label(
    header_frame,
    text=f" Student Details ({len(data_to_show)} students)",
    font=("Segoe UI", 14, "bold"),
    bg="#34495e",
    fg="white"
).pack()
tree_frame = tk.Frame(table_frame, bg="white")
tree_frame.pack(fill=tk.BOTH, expand=True)
columns = list(data_to_show.columns)
tree = ttk.Treeview(tree_frame, columns=columns, show="headings", height=15)
for col in columns:
    tree.heading(col, text=col)
    tree.column(col, width=120, minwidth=100)
for _, row in data_to_show.iterrows():
    tree.insert("", tk.END, values=list(row))
v_scrollbar = ttk.Scrollbar(tree_frame, orient=tk.VERTICAL, command=tree.yview)
h_scrollbar = ttk.Scrollbar(tree_frame, orient=tk.HORIZONTAL, command=tree.xview)
tree.configure(yscrollcommand=v_scrollbar.set, xscrollcommand=h_scrollbar.set)
tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
v_scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
h_scrollbar.pack(side=tk.BOTTOM, fill=tk.X)
table_frame.tree = tree

def show_analytics(self):
    if self.student_df is None:
        messagebox.showwarning("Warning", "Student data is not loaded. Cannot show analytics.")
        return
    analytics_window = tk.Toplevel(self.root)
    analytics_window.title(" Class Analytics Dashboard")
    analytics_window.geometry("900x700")
    analytics_window.configure(bg="white")
    header = tk.Label( analytics_window, text=" Class Analytics Dashboard",
                        font=("Segoe UI", 18, "bold"),
                        bg="#2c3e50",
                        fg="white",
                        pady=15
    )
    header.pack(fill=tk.X)
    notebook = ttk.Notebook(analytics_window)
    notebook.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

```

```

self.create_branch_analysis_tab(notebook)
self.create_performance_analysis_tab(notebook)
self.create_attendance_analysis_tab(notebook)

def create_branch_analysis_tab(self, notebook):
    branch_frame = tk.Frame(notebook, bg="white")
    notebook.add(branch_frame, text=" Branch Analysis")
    if 'Branch' in self.student_df.columns and 'Year' in self.student_df.columns:
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
        branch_counts = self.student_df['Branch'].value_counts()
        ax1.pie(branch_counts.values, labels=branch_counts.index, autopct='%1.1f%%')
        ax1.set_title('Branch Distribution')
        branch_year = pd.crosstab(self.student_df['Branch'], self.student_df['Year'])
        branch_year.plot(kind='bar', ax=ax2)
        ax2.set_title('Students per Branch by Year')
        ax2.set_xlabel('Branch')
        ax2.set_ylabel('Number of Students')
        ax2.legend(title='Year')
        plt.tight_layout()
        canvas = FigureCanvasTkAgg(fig, branch_frame)
        canvas.draw()
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
    else:
        info_label = tk.Label(
            branch_frame,
            text="Branch or Year data not found in the dataset for analysis.",
            font=("Segoe UI", 12),
            bg="white",
            fg="#2c3e50",
            justify=tk.CENTER
        )
        info_label.pack(expand=True)

def create_performance_analysis_tab(self, notebook):
    perf_frame = tk.Frame(notebook, bg="white")
    notebook.add(perf_frame, text=" Performance")
    if 'GPA' in self.student_df.columns:
        fig, ax = plt.subplots(figsize=(8, 5))
        sns.histplot(self.student_df['GPA'].dropna(), bins=10, kde=True, ax=ax)
        ax.set_title('GPA Distribution')
        ax.set_xlabel('GPA')
        ax.set_ylabel('Number of Students')
        canvas = FigureCanvasTkAgg(fig, perf_frame)
        canvas.draw()
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
    else:
        info_label = tk.Label(
            perf_frame,
            text=" Performance Analysis\n\nThis section will show GPA distributions,\n\npass/fail rates, and semester-wise trends.\n\n(GPA column not found in data)",
            font=("Segoe UI", 12),

```

```

        bg="white",
        fg="#2c3e50",
        justify=tk.CENTER
    )
    info_label.pack(expand=True)

def create_attendance_analysis_tab(self, notebook):
    attend_frame = tk.Frame(notebook, bg="white")
    notebook.add(attend_frame, text=" Attendance")
    if 'Overall Attendance (%)' in self.student_df.columns:
        fig, ax = plt.subplots(figsize=(10, 6))
        attendance_data = self.student_df['Overall Attendance (%)'].dropna()
        ax.hist(attendance_data, bins=20, alpha=0.7, color='skyblue', edgecolor='black')
        ax.set_title('Attendance Distribution')
        ax.set_xlabel('Attendance Percentage')
        ax.set_ylabel('Number of Students')
        ax.grid(True, alpha=0.3)
        plt.tight_layout()
        canvas = FigureCanvasTkAgg(fig, attend_frame)
        canvas.draw()
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
    else:
        info_label = tk.Label(
            attend_frame,
            text=" Attendance Analysis\n\nAttendance data not found in the dataset.",
            font=("Segoe UI", 12),
            bg="white",
            fg="#2c3e50",
            justify=tk.CENTER
        )
        info_label.pack(expand=True)

def export_data(self):
    if self.student_df is None:
        messagebox.showwarning("Warning", "Student data is not loaded. Cannot export data.")
        return
    data_to_export = self.filtered_df if self.filtered_df is not None else self.student_df
    file_path = filedialog.asksaveasfilename(
        title="Save Filtered Data",
        defaultextension=".xlsx",
        filetypes=[("Excel files", "*.xlsx")]
    )
    if file_path:
        try:
            data_to_export.to_excel(file_path, index=False)
            messagebox.showinfo("Success", f"Data exported successfully to {file_path}")
        except Exception as e:
            messagebox.showerror("Error", f"Failed to export data: {str(e)}")

def open_class_incharge_page():
    win = tk.Toplevel(root)
    ClassInchargeApp(win)

```

```

# ===== TIMETABLE POPUP =====
def show_timetable(faculty_name):
    timetable_window = tk.Toplevel(root)
    timetable_window.title(f'{faculty_name}'s Timetable")
    timetable_window.geometry("1200x700")
    timetable_window.configure(bg='#f8f9fa')
    header_frame = tk.Frame(timetable_window, bg='#2c3e50', height=80)
    header_frame.pack(fill='x')
    header_frame.pack_propagate(False)
    try:
        subject = faculty_df.loc[faculty_df["Faculty Name"] == faculty_name, "Subject"].values[0]
        sheet_name = f'{faculty_name}({subject.upper()})'
        header_content = tk.Frame(header_frame, bg='#2c3e50')
        header_content.pack(expand=True, fill='both', padx=30, pady=15)
        title_label = tk.Label(header_content, text=f'{faculty_name}'s Timetable",
                               font=("Segoe UI", 24, "bold"), bg='#2c3e50', fg='white')
        title_label.pack(side='left', anchor='w')
        subject_label = tk.Label(header_content, text=f'Subject: {subject}',
                                font=("Segoe UI", 14), bg='#2c3e50', fg='#ecf0f1')
        subject_label.pack(side='right', anchor='e')
        timetable_df = pd.read_excel(timetable_data_path, sheet_name=sheet_name)
        if timetable_df.empty:
            no_data_frame = tk.Frame(timetable_window, bg='#f8f9fa')
            no_data_frame.pack(expand=True, fill='both')
            no_data_label = tk.Label(no_data_frame, text=" No timetable data found",
                                    font=("Segoe UI", 18), bg='#f8f9fa', fg='#e74c3c')
            no_data_label.pack(expand=True)
            return
        content_frame = tk.Frame(timetable_window, bg='#f8f9fa')
        content_frame.pack(fill='both', expand=True, padx=30, pady=20)
        style = ttk.Style()
        style.theme_use('clam')
        style.configure("Modern.Treeview.Heading",
                        background="#3498db", foreground="white", font=("Segoe UI", 12, "bold"), relief="flat")
        style.configure("Modern.Treeview",
                        background="white", foreground="#2c3e50", rowheight=40, font=("Segoe UI", 11),
                        fieldbackground="white", borderwidth=1, relief="flat")
        style.map("Modern.Treeview",
                 background=[('selected', '#e3f2fd')],
                 foreground=[('selected', '#1976d2')])
        tree_frame = tk.Frame(content_frame, bg='white', relief='solid', bd=1)
        tree_frame.pack(fill='both', expand=True)
        tree = ttk.Treeview(tree_frame, style="Modern.Treeview")
        style.configure("Modern.Vertical.TScrollbar", background="#bdc3c7",
                        troughcolor="#ecf0f1", borderwidth=0, arrowcolor="#34495e")
        style.configure("Modern.Horizontal.TScrollbar", background="#bdc3c7",
                        troughcolor="#ecf0f1", borderwidth=0, arrowcolor="#34495e")
        v_scrollbar = ttk.Scrollbar(tree_frame, orient=tk.VERTICAL, command=tree.yview,
                                    style="Modern.Vertical.TScrollbar")
        h_scrollbar = ttk.Scrollbar(tree_frame, orient=tk.HORIZONTAL, command=tree.xview,

```

```

        style="Modern.Horizontal.TScrollbar")
tree.configure(yscrollcommand=v_scrollbar.set, xscrollcommand=h_scrollbar.set)
columns = list(timetable_df.columns)
tree["columns"] = columns
tree["show"] = "headings"
for i, col in enumerate(columns):
    tree.heading(col, text=col, anchor="center")
    if col == "Day":
        tree.column(col, width=150, anchor="center")
    else:
        tree.column(col, width=180, anchor="center")
for index, row in timetable_df.iterrows():
    values = []
    for col in columns:
        value = row[col]
        if pd.isna(value):
            values.append("—")
        else:
            values.append(str(value))
    tag = 'evenrow' if index % 2 == 0 else 'oddrow'
    tree.insert("", "end", values=values, tags=(tag,))
tree.tag_configure('evenrow', background='#ffffff')
tree.tag_configure('oddrow', background='#f8f9fa')
tree.grid(row=0, column=0, sticky="nsew")
v_scrollbar.grid(row=0, column=1, sticky="ns")
h_scrollbar.grid(row=1, column=0, sticky="ew")
tree_frame.grid_rowconfigure(0, weight=1)
tree_frame.grid_columnconfigure(0, weight=1)
button_frame = tk.Frame(timetable_window, bg='#f8f9fa', height=60)
button_frame.pack(fill='x', side='bottom')
button_frame.pack_propagate(False)

close_button = tk.Button(button_frame, text="× Close", command=timetable_window.destroy,
                          font=("Segoe UI", 12, "bold"), bg="#e74c3c", fg="white", padx=30, pady=10, relief="flat", bd=0,
cursor="hand2")
close_button.pack(pady=15)
except Exception as e:
    messagebox.showerror("Error", f"Could not load timetable: {e}")
timetable_window.destroy()

# ===== ACADEMIC LIST POPUP =====
def show_academic_list_popup():
    win = tk.Toplevel(root)
    win.title(" Academic List and Performance Analysis")
    win.geometry("1400x900")
    win.configure(bg='#f0f8ff')
    control_frame = tk.Frame(win, bg='#e0f2fe', relief='raised', bd=2)
    control_frame.pack(pady=15, padx=20, fill='x')
    view_var = tk.StringVar()
    year_var = tk.StringVar()
    branch_var = tk.StringVar()
    tk.Label(control_frame, text=" Select View:", font=("Arial", 11, "bold"),

```

```

        bg='#e0f2fe', fg='#1565c0').grid(row=0, column=0, padx=10, pady=10, sticky='e')
view_menu = ttk.Combobox(control_frame, textvariable=view_var, values=["Pass List", "Fail List"],
        state="readonly", font=("Arial", 10), width=15)
view_menu.grid(row=0, column=1, padx=10, pady=10, sticky='w')
tk.Label(control_frame, text=" Select Year:", font=("Arial", 11, "bold"),
        bg='#e0f2fe', fg='#1565c0').grid(row=0, column=2, padx=10, pady=10, sticky='e')
years = sorted(student_df['Year'].dropna().unique().astype(str).tolist())
years.insert(0, "All Years")
year_menu = ttk.Combobox(control_frame, textvariable=year_var, values=years,
        state="readonly", font=("Arial", 10), width=15)
year_menu.grid(row=0, column=3, padx=10, pady=10, sticky='w')
tk.Label(control_frame, text=" Select Branch:", font=("Arial", 11, "bold"),
        bg='#e0f2fe', fg='#1565c0').grid(row=0, column=4, padx=10, pady=10, sticky='e')
branches = sorted(student_df['Branch'].dropna().unique().tolist())
branches.insert(0, "All Branches")
branch_menu = ttk.Combobox(control_frame, textvariable=branch_var, values=branches,
        state="readonly", font=("Arial", 10), width=15)
branch_menu.grid(row=0, column=5, padx=10, pady=10, sticky='w')
button_frame = tk.Frame(win, bg='#f0f8ff')
button_frame.pack(pady=10)
main_content = tk.Frame(win, bg='#f0f8ff')
main_content.pack(fill='both', expand=True, padx=20, pady=10)
table_frame = tk.Frame(main_content, bg='#f0f8ff', width=500)
table_frame.pack(side='left', fill='y', padx=(0, 10))
table_frame.pack_propagate(False)
table_header = tk.Label(table_frame, text=" STUDENT RECORDS",
        font=("Arial", 12, "bold"), fg='#1565c0', bg='#f0f8ff')
table_header.pack(pady=(0, 10))
tree_container = tk.Frame(table_frame, bg='white', relief='sunken', bd=2)
tree_container.pack(fill='both', expand=True)
tree_scroll = tk.Scrollbar(tree_container)
tree_scroll.pack(side=tk.RIGHT, fill=tk.Y)
style = ttk.Style()
style.theme_use("clam")
style.configure("Treeview",
        background="white",
        foreground="black",
        rowheight=20,
        fieldbackground="white",
        font=("Arial", 9))
style.configure("Treeview.Heading",
        background="#1976d2",
        foreground="white",
        font=("Arial", 10, "bold"))
tree = ttk.Treeview(tree_container, yscrollcommand=tree_scroll.set,
        columns=("Name", "ROLLNO", "Branch", "Year", "Percentage"),
        show="headings")
headers = {"Name": " Name", "ROLLNO": " Roll No",
        "Branch": " Branch", "Year": " Year", "Percentage": " %"}
widths = {"Name": 120, "ROLLNO": 80, "Branch": 80, "Year": 50, "Percentage": 70}
for col in tree["columns"]:

```



```

tree.heading(col, text=headers[col])
tree.column(col, width=widths[col], anchor='center')
tree.pack(fill='both', expand=True, padx=5, pady=5)
tree_scroll.config(command=tree.yview)
chart_frame = tk.Frame(main_content, bg='#f8f8fc', relief='raised', bd=2)
chart_frame.pack(side='right', fill='both', expand=True)
chart_header = tk.Label(chart_frame, text=" PERFORMANCE VISUALIZATION",
                        font=("Arial", 12, "bold"), fg="#1565c0", bg='#f8f8fc')
chart_header.pack(pady=(10, 5))
fig, ax = plt.subplots(figsize=(12, 6))
fig.patch.set_facecolor('#f8f8fc')
canvas = FigureCanvasTkAgg(fig, master=chart_frame)
canvas.get_tk_widget().pack(fill="both", expand=True, padx=10, pady=10)
def filter_data(view_type, year, branch):
    df = student_df.copy()
    if year != "All Years" and year.strip() != "":
        try:
            df = df[df['Year'] == int(year)]
        except ValueError:
            return pd.DataFrame(columns=['Name', 'ROLLNO', 'Branch', 'Year', 'Percentage'])
    if branch != "All Branches" and branch.strip() != "":
        df = df[df['Branch'] == branch]
    view_type = view_type.strip()
    if view_type == "Pass List":
        df = df[df['Percentage'].notna() & (df['Percentage'] >= 50)]
    elif view_type == "Fail List":
        df = df[df['Percentage'].isna() | (df['Percentage'] < 50)]
    return df[['Name', 'ROLLNO', 'Branch', 'Year', 'Percentage']]
def update_view():
    view_type = view_var.get()
    year = year_var.get()
    branch = branch_var.get()
    if not view_type or not year or not branch:
        return
    filtered = filter_data(view_type, year, branch)
    for row in tree.get_children():
        tree.delete(row)
    if filtered.empty:
        tree.insert("", "end", values=("No data found", "", "", "", ""))
        ax.clear()
        ax.set_facecolor('ffffff')
        ax.text(0.5, 0.5, "No data to display", ha='center', va='center', fontsize=14, color='gray')
        ax.set_xticks([])
        ax.set_yticks([])
        canvas.draw()
        return
    for index, row_ in filtered.iterrows():
        tree.insert("", "end", values=list(row_))
    ax.clear()
    fig.patch.set_facecolor('#f0f8ff')
    ax.set_facecolor('ffffff')

```

```

if branch != "All Branches":
    if view_type.strip() == "Fail List":
        plot_data = filtered.copy()
        plot_data['Percentage'] = plot_data['Percentage'].fillna(0)
        y_positions = range(len(plot_data))
        colors = []
        for pct in plot_data['Percentage']:
            if pct == 0:
                colors.append('#FF0000')
            elif pct < 25:
                colors.append('#FF4500')
            elif pct < 40:
                colors.append('#FF6B35')
            elif pct < 50:
                colors.append('#FF8C42')
            else:
                colors.append('#FFA500')
        bars = ax.barh(y_positions, plot_data['Percentage'],
                       color=colors, edgecolor='black', linewidth=2, alpha=0.8, height=0.7)
        for i, (bar, val, name) in enumerate(zip(bars, plot_data['Percentage'], plot_data['Name'])):
            width = bar.get_width()
            if val == 0:
                label_text = 'ABSENT (0%)'
                label_color = 'white'
                x_pos = 5
            else:
                label_text = f'{val:.1f}%'
                label_color = 'white' if val < 25 else 'black'
                x_pos = width + 1
            ax.text(x_pos, bar.get_y() + bar.get_height()/2,
                   label_text, ha='left', va='center',
                   fontweight='bold', fontsize=11, color=label_color,
                   bbox=dict(boxstyle="round,pad=0.3", facecolor='black' if val == 0 else 'white',
                             alpha=0.8, edgecolor='black'))
        ax.set_yticks(y_positions)
        ax.set_yticklabels(plot_data['Name'], fontsize=10)
        ax.set_xlim(0, max(60, plot_data['Percentage'].max() + 20))
        ax.axvline(x=50, color='red', linestyle='--', linewidth=3, alpha=0.8, label='Pass Mark (50%)')
        ax.axvline(x=35, color='darkred', linestyle=':', linewidth=2, alpha=0.6, label='Critical (35%)')
        from matplotlib.patches import Patch
        legend_elements = [
            Patch(facecolor='#FF0000', alpha=0.8, label='Absent (0%)'),
            Patch(facecolor='#FF4500', alpha=0.8, label='Very Poor (<25%)'),
            Patch(facecolor='#FF6B35', alpha=0.8, label='Poor (25-40%)'),
            Patch(facecolor='#FF8C42', alpha=0.8, label='Below Pass (40-50%)'),
            plt.Line2D([0], [0], color='red', linestyle='--', linewidth=2, label='Pass Mark (50%)')
        ]
        ax.legend(handles=legend_elements, loc='lower right', fontsize=9)
        ax.set_xlabel("Percentage (%)", fontweight='bold', fontsize=12)
        ax.set_ylabel("Student Names", fontweight='bold', fontsize=12)
        ax.set_title(" FAIL LIST - Individual Performance Analysis",

```

```

        fontweight='bold', fontsize=14, color='#8B0000', pad=20)
else:
    y_positions = range(len(filtered))
    colors = []
    grades = []
    for pct in filtered['Percentage']:
        if pct >= 90:
            colors.append('#1B5E20')
            grades.append('A+')
        elif pct >= 80:
            colors.append('#2E7D32')
            grades.append('A')
        elif pct >= 70:
            colors.append('#388E3C')
            grades.append('B+')
        elif pct >= 60:
            colors.append('#43A047')
            grades.append('B')
        elif pct >= 50:
            colors.append('#66BB6A')
            grades.append('C')
        else:
            colors.append('#FFC107')
            grades.append('D')
    bars = ax.barh(y_positions, filtered['Percentage'],
                   color=colors, edgecolor='black', linewidth=1.5, alpha=0.9, height=0.7)
    for i, (bar, val, grade, name) in enumerate(zip(bars, filtered['Percentage'], grades, filtered['Name'])):
        width = bar.get_width()
        ax.text(width + 1, bar.get_y() + bar.get_height()/2,
                f'{val:.1f}% ({grade})', ha='left', va='center',
                fontweight='bold', fontsize=10, color='black',
                bbox=dict(boxstyle="round,pad=0.3", facecolor='white', alpha=0.9, edgecolor='black'))
    ax.set_yticks(y_positions)
    ax.set_yticklabels(filtered['Name'], fontsize=10)
    ax.set_xlim(0, max(105, filtered['Percentage'].max() + 15))
    ax.axvline(x=50, color='green', linestyle='--', linewidth=3, alpha=0.8, label='Pass Mark (50%)')
    ax.axvline(x=75, color='blue', linestyle=':', linewidth=2, alpha=0.6, label='Distinction (75%)')
    from matplotlib.patches import Patch
    legend_elements = [
        Patch(facecolor='#1B5E20', alpha=0.9, label='Excellent (90%+)'),
        Patch(facecolor='#2E7D32', alpha=0.9, label='Very Good (80-90%)'),
        Patch(facecolor='#388E3C', alpha=0.9, label='Good (70-80%)'),
        Patch(facecolor='#43A047', alpha=0.9, label='Average (60-70%)'),
        Patch(facecolor='#66BB6A', alpha=0.9, label='Pass (50-60%)'),
        plt.Line2D([0], [0], color='green', linestyle='--', linewidth=2, label='Pass Mark (50%)')
    ]
    ax.legend(handles=legend_elements, loc='lower right', fontsize=9)
    ax.set_xlabel("Percentage (%)", fontweight='bold', fontsize=12)
    ax.set_ylabel("Student Names", fontweight='bold', fontsize=12)
    ax.set_title(" PASS LIST - Individual Performance Analysis",
                 fontweight='bold', fontsize=14, color='#1B5E20', pad=20)

```

```

else:
    branches_unique = filtered['Branch'].unique()
    if view_type.strip() == "Fail List":
        plot_data = filtered.copy()
        plot_data['Percentage'] = plot_data['Percentage'].fillna(0)
        fail_categories = {
            'Absent': [],
            'Very Poor (<25%): [],
            'Poor (25-40%): [],
            'Below Pass (40-50%): []
        }
        branch_labels = []
        for branch in branches_unique:
            branch_data = plot_data[plot_data['Branch'] == branch]
            branch_labels.append(branch)
            absent = len(branch_data[branch_data['Percentage'] == 0])
            very_poor = len(branch_data[(branch_data['Percentage'] > 0) & (branch_data['Percentage'] < 25)])
            poor = len(branch_data[(branch_data['Percentage'] >= 25) & (branch_data['Percentage'] < 40)])
            below_pass = len(branch_data[(branch_data['Percentage'] >= 40) & (branch_data['Percentage'] < 50)])
            fail_categories['Absent'].append(absent)
            fail_categories['Very Poor (<25%)'].append(very_poor)
            fail_categories['Poor (25-40%)'].append(poor)
            fail_categories['Below Pass (40-50%)'].append(below_pass)
        x = np.arange(len(branch_labels))
        width = 0.6
        colors = ['#8B0000', '#DC143C', '#FF6347', '#FF8C42']
        bottom = np.zeros(len(branch_labels))
        for i, (category, values) in enumerate(fail_categories.items()):
            bars = ax.bar(x, values, width, label=category, color=colors[i],
                          bottom=bottom, alpha=0.8, edgecolor='black', linewidth=1)
            for j, (bar, val) in enumerate(zip(bars, values)):
                if val > 0:
                    ax.text(bar.get_x() + bar.get_width()/2, bottom[j] + val/2,
                           str(val), ha='center', va='center', fontweight='bold',
                           color='white', fontsize=10)
            bottom += values
        ax.set_xlabel("Branch", fontweight='bold', fontsize=12)
        ax.set_ylabel("Number of Students", fontweight='bold', fontsize=12)
        ax.set_title(" FAIL LIST - Branch-wise Distribution by Categories",
                     fontweight='bold', fontsize=14, color='#8B0000', pad=20)
        ax.set_xticks(x)
        ax.set_xticklabels(branch_labels, rotation=45, ha='right')
        ax.legend(loc='upper right')
    else:
        grade_categories = {
            'Excellent (90%+): [],
            'Very Good (80-90%): [],
            'Good (70-80%): [],
            'Average (60-70%): [],
            'Pass (50-60%): []
        }

```

```

branch_labels = []
for branch in branches_unique:
    branch_data = filtered[filtered['Branch'] == branch]
    branch_labels.append(branch)
    excellent = len(branch_data[branch_data['Percentage'] >= 90])
    very_good = len(branch_data[(branch_data['Percentage'] >= 80) & (branch_data['Percentage'] < 90)])
    good = len(branch_data[(branch_data['Percentage'] >= 70) & (branch_data['Percentage'] < 80)])
    average = len(branch_data[(branch_data['Percentage'] >= 60) & (branch_data['Percentage'] < 70)])
    just_pass = len(branch_data[(branch_data['Percentage'] >= 50) & (branch_data['Percentage'] < 60)])
    grade_categories['Excellent (90%+)'].append(excellent)
    grade_categories['Very Good (80-90%)'].append(very_good)
    grade_categories['Good (70-80%)'].append(good)
    grade_categories['Average (60-70%)'].append(average)
    grade_categories['Pass (50-60%)'].append(just_pass)
x = np.arange(len(branch_labels))
width = 0.6
colors = ['#1B5E20', '#2E7D32', '#388E3C', '#43A047', '#66BB6A']
bottom = np.zeros(len(branch_labels))
for i, (category, values) in enumerate(grade_categories.items()):
    bars = ax.bar(x, values, width, label=category, color=colors[i],
                  bottom=bottom, alpha=0.9, edgecolor='black', linewidth=1)
    for j, (bar, val) in enumerate(zip(bars, values)):
        if val > 0:
            ax.text(bar.get_x() + bar.get_width()/2, bottom[j] + val/2,
                    str(val), ha='center', va='center', fontweight='bold',
                    color='white', fontsize=10)
    bottom += values
ax.set_xlabel("Branch", fontweight='bold', fontsize=12)
ax.set_ylabel("Number of Students", fontweight='bold', fontsize=12)
ax.set_title(" PASS LIST - Branch-wise Grade Distribution",
             fontweight='bold', fontsize=14, color='#1B5E20', pad=20)
ax.set_xticks(x)
ax.set_xticklabels(branch_labels, rotation=45, ha='right')
ax.legend(loc='upper right')
ax.grid(True, alpha=0.3, linestyle='-', linewidth=0.5)
ax.set_axisbelow(True)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_color('#666666')
ax.spines['bottom'].set_color('#666666')
canvas.draw()
tk.Button(button_frame, text=" ANALYZE DATA", command=update_view,
          bg="#1976d2", fg="white", font=("Arial", 12, "bold"),
          relief='raised', bd=3, padx=20, pady=8,
          activebackground="#1565c0", activeforeground="white").pack()
ax.clear()
ax.set_facecolor('ffffff')
ax.text(0.5, 0.5, " Select filters and click 'ANALYZE DATA' to view performance charts",
       ha='center', va='center', fontsize=14, color='#6b7280',
       bbox=dict(boxstyle="round,pad=0.5", facecolor='#e5e7eb', alpha=0.8))
ax.set_xticks([])

```

```

ax.set_yticks([])
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(False)
canvas.draw()

# ===== MAIN TK ROOT =====
root = tk.Tk()
root.title("College Management System")
root.geometry("950x700")
root.configure(bg="#ecf0f1")

# ===== PAGE FUNCTIONS =====
def show_home_page():
    for widget in root.winfo_children():
        widget.destroy()
    root.configure(bg="#ecf0f1")
    heading = tk.Label(root, text="COLLEGE MANAGEMENT SYSTEM", font=(
        "Segoe UI", 22, "bold"), bg="#2c3e50", fg="white", pady=20)
    heading.pack(fill=tk.X)
    panel = tk.Frame(root, bg="white", bd=0)
    panel.place(relx=0.5, rely=0.5, anchor=tk.CENTER, width=300, height=180)
    tk.Label(panel, text="Choose Panel", font=("Segoe UI", 14, "bold"),
        bg="white", fg="#2c3e50", pady=10).pack()
    btn_style = {
        "font": ("Segoe UI", 12, "bold"),
        "bg": "#2980b9",
        "fg": "white",
        "activebackground": "#1f618d",
        "activeforeground": "white",
        "width": 20,
        "relief": tk.FLAT,
        "cursor": "hand2",
        "bd": 0,
        "pady": 5
    }
    tk.Button(panel, text="Student Panel", command=show_student_panel, **btn_style).pack(pady=5)
    tk.Button(panel, text="Faculty Panel", command=show_faculty_panel, **btn_style).pack(pady=5)

def show_faculty_panel():
    for widget in root.winfo_children():
        widget.destroy()
    root.configure(bg="#f0f4f7")
    header = tk.Label(root, text="College Management System", font=("Segoe UI", 18, "bold"), bg="#34495e",
        fg="white", padx=20, pady=10, anchor="w")
    header.pack(fill=tk.X)
    dialog = tk.Frame(root, bg="white", bd=1, relief=tk.FLAT)
    dialog.place(relx=0.5, rely=0.5, anchor=tk.CENTER, width=350, height=220)
    tk.Label(dialog, text="Login", font=("Segoe UI", 16, "bold"),
        bg="white", fg="#2c3e50").pack(pady=10)

```

```

tk.Label(dialog, text="Username", bg="white", anchor="w",
        font=("Segoe UI", 10)).pack(fill="x", padx=20)
username_entry = tk.Entry(dialog, font=("Segoe UI", 11), bd=1, relief=tk.SOLID)
username_entry.pack(fill="x", padx=20, pady=5)
tk.Label(dialog, text="Password", bg="white", anchor="w",
        font=("Segoe UI", 10)).pack(fill="x", padx=20)
password_entry = tk.Entry(dialog, show="*", font=("Segoe UI", 11), bd=1, relief=tk.SOLID)
password_entry.pack(fill="x", padx=20, pady=5)
def login_action():
    username = username_entry.get()
    password = password_entry.get()
    for _, row in faculty_df.iterrows():
        if str(row['Username']).strip() == username and str(row['Password']).strip() == password:
            show_faculty_dashboard(row)
            return
    messagebox.showerror("Login Failed", "Invalid username or password.")
tk.Button(dialog, text="Login", font=("Segoe UI", 12, "bold"), bg="#27ae60", fg="white",
        activebackground="#219150", activeforeground="white", command=login_action,
        relief=tk.FLAT).pack(pady=10)

def show_faculty_dashboard(faculty_row):
    for widget in root.winfo_children():
        widget.destroy()
    root.configure(bg="#f0f4f7")
    header = tk.Label(root, text="College Management System", font=("Segoe UI", 18, "bold"),
        bg="#34495e", fg="white", padx=20, pady=10, anchor="w")
    header.pack(fill=tk.X)
    menu_frame = tk.Frame(root, bg="#34495e", height=50)
    menu_frame.pack(fill=tk.X)
    button_style = {
        "font": ("Segoe UI", 12, "bold"),
        "bg": "#2980b9",
        "fg": "white",
        "activebackground": "#1f618d",
        "activeforeground": "white",
        "padx": 10,
        "pady": 5,
        "bd": 0,
        "cursor": "hand2"
    }
    #student_details_btn = tk.Button(menu_frame, text="Student Details", **button_style, command=lambda:
messagebox.showinfo("Student Details", "Not implemented in this demo.))
    #student_details_btn.pack(side=tk.LEFT, padx=5, pady=5)
    #academic_dashboard_btn = tk.Button(menu_frame, text="Academic Dashboard", **button_style, command=lambda:
messagebox.showinfo("Academic Dashboard", "Not implemented in this demo.))
    #academic_dashboard_btn.pack(side=tk.LEFT, padx=5, pady=5)
    class_incharge_btn = tk.Button(menu_frame, text="Class Incharge", **button_style, command=lambda:
open_class_incharge_page())
    class_incharge_btn.pack(side=tk.LEFT, padx=5, pady=5)
    faculty_name = faculty_row["Faculty Name"]
    faculty_timetable_btn = tk.Button(menu_frame, text="Faculty Timetable", **button_style,

```

```

        command=lambda: show_timetable(faculty_name))
faculty_timetable_btn.pack(side=tk.LEFT, padx=5, pady=5)
academic_list_btn = tk.Button(menu_frame, text="Academic List", **button_style, command=show_academic_list_popup)
academic_list_btn.pack(side=tk.LEFT, padx=5, pady=5)
tk.Button(menu_frame, text="Logout/Home", **button_style, command=show_home_page).pack(side=tk.RIGHT, padx=5)
academic_dashboard_btn = tk.Button(menu_frame, text="Academic Dashboard", **button_style,
command=show_academic_dashboard_popup)
academic_dashboard_btn.pack(side=tk.LEFT, padx=5, pady=5)

def show_student_panel():
    for widget in root.winfo_children():
        widget.destroy()
    header = tk.Label(root, text="Student Panel", font=("Segoe UI", 18, "bold"), bg="#34495e",
        fg="white", padx=20, pady=10, anchor="w")
    header.pack(fill=tk.X)
    dialog = tk.Frame(root, bg="white", bd=1, relief=tk.FLAT)
    dialog.place(relx=0.5, rely=0.5, anchor=tk.CENTER, width=350, height=220)
    tk.Label(dialog, text="Login", font=("Segoe UI", 16, "bold"),
        bg="white", fg="#2c3e50").pack(pady=10)
    tk.Label(dialog, text="Username", bg="white", anchor="w",
        font=("Segoe UI", 10)).pack(fill="x", padx=20)
    username_entry = tk.Entry(dialog, font=("Segoe UI", 11), bd=1, relief=tk.SOLID)
    username_entry.pack(fill="x", padx=20, pady=5)
    tk.Label(dialog, text="Password", bg="white", anchor="w",
        font=("Segoe UI", 10)).pack(fill="x", padx=20)
    password_entry = tk.Entry(dialog, show="*", font=("Segoe UI", 11), bd=1, relief=tk.SOLID)
    password_entry.pack(fill="x", padx=20, pady=5)
    def login_action():
        username = username_entry.get()
        password = password_entry.get()
        for _, row in student_df.iterrows():
            if str(row['user name']).strip() == username and str(row['Password']).strip() == password:
                show_student_dashboard(row)
                return
        messagebox.showerror("Login Failed", "Invalid username or password.")
    tk.Button(dialog, text="Login", font=("Segoe UI", 12, "bold"), bg="#27ae60", fg="white",
        activebackground="#219150", activeforeground="white", command=login_action,
        relief=tk.FLAT).pack(pady=10)

def show_academic_dashboard_popup():
    import pandas as pd
    import matplotlib.pyplot as plt
    import numpy as np
    import tkinter as tk
    from tkinter import ttk, messagebox
    from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

    # Load your dataset with updated file path
    df = pd.read_excel(r"C:\Users\admin\Downloads\Updated_Student_Data_With_Fails (1).xlsx")
    #D:\PROJECT 6PHRASE\new data.xlsx
    df.fillna("-", inplace=True)

```



```

df["Year"] = df["Year"].astype(str)

dash = tk.Toplevel(root)
dash.title("Student Academic Dashboard")
dash.geometry("1200x800")
dash.configure(bg="#e6f0ff")

title_label = tk.Label(dash, text="Student Academic Dashboard",
                        font=("Helvetica", 20, "bold"),
                        bg="#003366", fg="white")
title_label.pack(fill=tk.X)

frame_controls = tk.Frame(dash, bg="#e6f0ff")
frame_controls.pack(pady=10)

tk.Label(frame_controls, text="Year:", bg="#e6f0ff",
         font=("Helvetica", 12)).grid(row=0, column=0, padx=10)
year_var = tk.StringVar(value="1")
year_menu = ttk.Combobox(frame_controls, textvariable=year_var,
                          values=sorted(df["Year"].unique().tolist()),
                          width=10, state="readonly")
year_menu.grid(row=0, column=1)

tk.Label(frame_controls, text="Branch:", bg="#e6f0ff",
         font=("Helvetica", 12)).grid(row=0, column=2, padx=10)
branch_var = tk.StringVar(value="All")
branches = sorted(df["Branch"].unique().tolist())
branch_menu = ttk.Combobox(frame_controls, textvariable=branch_var,
                           values=["All"] + branches,
                           width=10, state="readonly")
branch_menu.grid(row=0, column=3)

current_tree = [None]
current_filtered_data = [None]

def show_student_details():
    if not current_tree[0] or current_filtered_data[0] is None:
        messagebox.showinfo("No Data", "Please load data first by clicking 'Show Data'")
        return
    try:
        selection = current_tree[0].selection()
        if not selection:
            messagebox.showwarning("No Selection", "Please select a student from the table first.")
            return
        selected_item = selection[0]
        values = current_tree[0].item(selected_item, 'values')
        student_name = values[0]
        student_data = current_filtered_data[0][current_filtered_data[0].iloc[:, 0] == student_name].iloc[0]
        # --- Details window below, as in your code ---
        details_window = tk.Toplevel(dash)
        details_window.title(f"Student Profile - {student_name}")

```

```

details_window.geometry("600x700")
details_window.configure(bg="#f8f9fa")
details_window.resizable(True, True)
details_window.transient(dash)
details_window.grab_set()
header_frame = tk.Frame(details_window, bg="#003366", height=70)
header_frame.pack(fill=tk.X)
header_frame.pack_propagate(False)
tk.Label(header_frame, text=" Student Academic Profile",
        font=("Helvetica", 16, "bold"),
        bg="#003366", fg="white").pack(pady=20)
name_frame = tk.Frame(details_window, bg="#e3f2fd", height=60)
name_frame.pack(fill=tk.X, padx=20, pady=(15, 10))
name_frame.pack_propagate(False)
tk.Label(name_frame, text=f" {student_name}",
        font=("Helvetica", 14, "bold"),
        bg="#e3f2fd", fg="#1565c0").pack(pady=15)
content_frame = tk.Frame(details_window, bg="#f8f9fa")
content_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=(0, 15))
canvas = tk.Canvas(content_frame, bg="#f8f9fa", highlightthickness=0)
scrollbar = ttk.Scrollbar(content_frame, orient="vertical", command=canvas.yview)
scrollable_frame = tk.Frame(canvas, bg="#f8f9fa")
scrollable_frame.bind("<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all")))
canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
canvas.configure(yscrollcommand=scrollbar.set)
for i, (field, value) in enumerate(student_data.items()):
    if field == student_data.index[0]: # Skip first field (student name)
        continue
    card_frame = tk.Frame(scrollable_frame, bg="white", relief="solid", bd=1)
    card_frame.pack(fill=tk.X, padx=10, pady=5)
    header_bg = "#f1f8e9" if "Pass" in str(value) else "#ffebee" if "Fail" in str(value) else "#e8f5e8"
    field_frame = tk.Frame(card_frame, bg=header_bg, height=35)
    field_frame.pack(fill=tk.X)
    field_frame.pack_propagate(False)
    icon = ""
    if "percentage" in field.lower() or "%" in field.lower():
        icon = " "
    elif "year" in field.lower():
        icon = " "
    elif "branch" in field.lower():
        icon = " "
    elif "roll" in field.lower() or "id" in field.lower():
        icon = " "
    elif "fail" in field.lower() or "pass" in field.lower():
        icon = " " if "Pass" in str(value) else " " if "Fail" in str(value) else " "
    else:
        icon = " "
    tk.Label(field_frame, text=f"{icon} {field}",
            font=("Helvetica", 10, "bold"),
            bg=header_bg, fg="#2e7d32").pack(anchor="w", padx=15, pady=8)

```

```

display_value = str(value) if str(value) not in ["nan", "-", ""] else "N/A"
value_color = "#2e7d32" if "Pass" in str(value) else "#d32f2f" if "Fail" in str(value) else "#424242"
value_bg = "white"
value_label = tk.Label(card_frame, text=display_value,
                        font=("Helvetica", 11),
                        bg=value_bg, fg=value_color,
                        wraplength=500, justify="left")
value_label.pack(anchor="w", padx=20, pady=(0, 15))
canvas.pack(side="left", fill="both", expand=True)
scrollbar.pack(side="right", fill="y")
button_frame = tk.Frame(details_window, bg="#f8f9fa")
button_frame.pack(fill=tk.X, pady=(10, 20))
def show_summary():
    summary_text = f" Academic Summary for {student_name}\n"
    summary_text += "="*50 + "\n\n"
    for field, value in student_data.items():
        if any(keyword in field.lower() for keyword in ['percentage', 'year', 'branch', 'pass', 'fail']):
            summary_text += f"• {field}: {value}\n"
    summary_window = tk.Toplevel(details_window)
    summary_window.title(f" Academic Summary - {student_name}")
    summary_window.geometry("500x400")
    summary_window.configure(bg="white")
    text_widget = tk.Text(summary_window, wrap=tk.WORD, font=("Consolas", 10),
                          bg="white", fg="#333333", padx=20, pady=20)
    text_widget.pack(fill=tk.BOTH, expand=True)
    text_widget.insert(tk.END, summary_text)
    text_widget.config(state=tk.DISABLED)
    summary_btn = tk.Button(button_frame, text=" Academic Summary",
                           command=show_summary,
                           bg="#2196f3", fg="white",
                           font=("Helvetica", 10, "bold"),
                           padx=20, pady=8)
    summary_btn.pack(side=tk.LEFT, padx=20)
    close_btn = tk.Button(button_frame, text=" Close",
                          command=details_window.destroy,
                          bg="#f44336", fg="white",
                          font=("Helvetica", 10, "bold"),
                          padx=20, pady=8)
    close_btn.pack(side=tk.RIGHT, padx=20)
    details_window.update_idletasks()
    x = (details_window.winfo_screenwidth() // 2) - (details_window.winfo_width() // 2)
    y = (details_window.winfo_screenheight() // 2) - (details_window.winfo_height() // 2)
    details_window.geometry(f"+{x}+{y}")
except Exception as e:
    messagebox.showerror("Error", f"An error occurred: {str(e)}")

def on_double_click(event):
    show_student_details()

def show_context_menu(event):
    if current_tree[0]:

```

```

try:
    item = current_tree[0].identify_row(event.y)
    if item:
        current_tree[0].selection_set(item)
        context_menu = tk.Menu(dash, tearoff=0)
        context_menu.add_command(label=" View Student Details",
                                command=show_student_details)
        context_menu.tk_popup(event.x_root, event.y_root)
except:
    pass

def show_data():
    selected_year = year_var.get()
    selected_branch = branch_var.get()
    filtered = df[df["Year"] == selected_year]
    if selected_branch != "All":
        filtered = filtered[filtered["Branch"] == selected_branch]
    current_filtered_data[0] = filtered
    for widget in tree_frame.winfo_children():
        widget.destroy()
    columns = list(filtered.columns)
    tree = ttk.Treeview(tree_frame, columns=columns, show="headings")
    style = ttk.Style()
    style.configure("Treeview", font=("Helvetica", 9))
    style.configure("Treeview.Heading", font=("Helvetica", 10, "bold"))
    tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    current_tree[0] = tree
    yscroll = tk.Scrollbar(tree_frame, orient="vertical", command=tree.yview)
    yscroll.pack(side=tk.RIGHT, fill=tk.Y)
    tree.configure(yscrollcommand=yscroll.set)
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=120, anchor=tk.CENTER, minwidth=80)
    for _, row in filtered.iterrows():
        row_data = [str(x) if str(x) not in ["nan", ""] else "-" for x in row.tolist()]
        tree.insert("", tk.END, values=row_data)
    tree.bind("<Double-1>", on_double_click)
    tree.bind("<Button-3>", show_context_menu)
    instruction_frame = tk.Frame(tree_frame, bg="#e6f0ff")
    instruction_frame.pack(fill=tk.X, pady=5)
    tk.Label(instruction_frame,
            text=" Double-click any student row to view detailed profile | Right-click for menu",
            bg="#e6f0ff", font=("Helvetica", 9, "italic"),
            fg="#666666").pack()
    show_pie_chart(filtered)

show_button = tk.Button(frame_controls, text="Show Data", command=show_data,
                        bg="#007acc", fg="white",
                        font=("Helvetica", 12, "bold"))
show_button.grid(row=0, column=4, padx=10)
details_button = tk.Button(frame_controls, text=" View Details",

```

```

        command=show_student_details,
        bg="#4caf50", fg="white",
        font=("Helvetica", 12, "bold"))
details_button.grid(row=0, column=5, padx=10)

tree_frame = tk.Frame(dash, bg="#e6f0ff")
tree_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
chart_frame = tk.Frame(dash, bg="#e6f0ff")
chart_frame.pack(pady=10)
fig = plt.Figure(figsize=(6, 4), dpi=100)
pie_canvas = FigureCanvasTkAgg(fig, master=chart_frame)
pie_canvas.get_tk_widget().pack()

def show_pie_chart(filtered_df):
    fig.clf()
    ax = fig.add_subplot(111)
    percentages = pd.to_numeric(filtered_df["Percentage"], errors='coerce').dropna()
    if len(percentages) == 0:
        ax.text(0.5, 0.5, 'No valid percentage data',
               horizontalalignment='center', verticalalignment='center',
               transform=ax.transAxes, fontsize=14)
    pie_canvas.draw()
    return
    bins = [0, 50, 75, 90, 100]
    labels = ["< 50%", "50-75%", "75-90%", "90-100%"]
    counts = pd.cut(percentages, bins=bins, labels=labels, right=False).value_counts().sort_index()
    counts = counts[counts > 0]
    if len(counts) > 0:
        explode = [0.05] * len(counts)
        colors = plt.cm.Pastel1.colors[:len(counts)]
        ax.pie(counts, labels=counts.index.tolist(), autopct="%1.1f%%",
              startangle=90, colors=colors, explode=explode)
        ax.set_title("Student Distribution by Percentage Range", fontsize=12)
        ax.axis("equal")
    pie_canvas.draw()

def show_student_dashboard(student_row):
    for widget in root.winfo_children():
        widget.destroy()
    root.configure(bg="#ecf0f1")
    header = tk.Label(root, text="College Management System", font=("Segoe UI", 18, "bold"),
                      bg="#2c3e50", fg="white", padx=20, pady=10, anchor="w")
    header.pack(fill=tk.X, anchor="nw")
    info_frame = tk.Frame(root, bg="#dfe6e9", pady=10)
    info_frame.pack(fill=tk.X, padx=20)
    left_info = tk.Frame(info_frame, bg="#dfe6e9")
    left_info.pack(side=tk.LEFT, expand=True, anchor="w")
    center_info = tk.Frame(info_frame, bg="#dfe6e9")
    center_info.pack(side=tk.LEFT, expand=True)
    right_info = tk.Frame(info_frame, bg="#dfe6e9")

```

```

right_info.pack(side=tk.RIGHT, expand=True, anchor="e")
name = student_row.get("Name", "N/A")
year = student_row.get("Year", "N/A")
roll = student_row.get("ROLLNO", "N/A")
branch = student_row.get("Branch", "N/A")
attendance = student_row.get("Overall Attendance (%)", "N/A")
percentage = student_row.get("Percentage", "N/A")
tk.Label(left_info, text=f"Name: {name}", font=("Segoe UI", 12, "bold"),
        bg="#dfe6e9", anchor="w").pack(anchor="w")
tk.Label(left_info, text=f"Year: {year}", font=("Segoe UI", 12),
        bg="#dfe6e9", anchor="w").pack(anchor="w")
tk.Label(center_info, text=f"Attendance: {attendance}%", font=("Segoe UI", 12, "bold"),
        bg="#dfe6e9", fg="#2c3e50").pack(anchor="center")
tk.Label(right_info, text=f"Roll No: {roll}", font=("Segoe UI", 12, "bold"),
        bg="#dfe6e9", anchor="e").pack(anchor="e")
tk.Label(right_info, text=f"Branch: {branch}", font=("Segoe UI", 12),
        bg="#dfe6e9", anchor="e").pack(anchor="e")
academic_frame = tk.Frame(root, bg="white", padx=10, pady=10)
academic_frame.pack(fill=tk.BOTH, padx=20, pady=10, expand=True)
table_frame = tk.Frame(academic_frame, bg="white")
table_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 10))
chart_frame = tk.Frame(academic_frame, bg="white")
chart_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
tk.Label(table_frame, text="Semester-wise GPA", font=("Segoe UI", 14, "bold"),
        bg="white", fg="#2c3e50", pady=10).pack(anchor="center")
header_row = tk.Frame(table_frame, bg="#f5f6fa", pady=5)
header_row.pack(fill=tk.X, pady=(0, 2))
tk.Label(header_row, text="Semester", font=("Segoe UI", 12, "bold"), bg="#f5f6fa",
        width=15, anchor="center").pack(side="left", padx=5)
tk.Label(header_row, text="GPA", font=("Segoe UI", 12, "bold"), bg="#f5f6fa",
        width=10, anchor="center").pack(side="left", padx=5)
for i in range(1, year * 2 + 1):
    sem_label = f"Sem{i}"
    gpa = student_row.get(f"{sem_label}(GPA)")
    if pd.isna(gpa):
        gpa = student_row.get(f"{sem_label}(CGPA)")
    row_bg = "#ffffff" if i % 2 == 0 else "#f9f9f9"
    row = tk.Frame(table_frame, bg=row_bg, pady=2)
    row.pack(fill=tk.X)
    tk.Label(row, text=sem_label, font=("Segoe UI", 11), bg=row_bg,
            width=15, anchor="center").pack(side="left", padx=5)
    tk.Label(row, text=gpa, font=("Segoe UI", 11), bg=row_bg,
            width=10, anchor="center").pack(side="left", padx=5)
chart_path = plot_gpa_chart(student_row)
chart_img = Image.open(chart_path).resize((500, 250))
chart_photo = ImageTk.PhotoImage(chart_img)
chart_label = tk.Label(chart_frame, image=chart_photo, bg="white")
chart_label.image = chart_photo
chart_label.pack()
percent_label = tk.Label(root, text=f"Overall Percentage: {percentage}%", font=("Segoe UI", 13, "bold"),
        bg="#ecf0f1", fg="#2c3e50", pady=10)

```

```
percent_label.pack()
```

```
show_home_page()
```

```
root.mainloop()
```

RESULTS

The final College Management System was tested with sample data sets and successfully demonstrated key functionalities. Faculty members could easily upload and manage student academic records, calculate GPA, and generate line/bar charts reflecting individual and batch performance. The Excel file integration worked seamlessly, allowing real-time updates to the dataset. For students, the system enabled viewing of their semester-wise results, cumulative GPA, and performance trends. The login authentication correctly separated access privileges. The charts helped faculty visually assess top-performing and underperforming students, aiding in academic planning. The system's GUI was found to be responsive and intuitive, even for non-technical users. Real-time updates and interactive chart filters were highlights of the user experience. The project thus met its core objectives of centralization, ease of access, and data visualization.

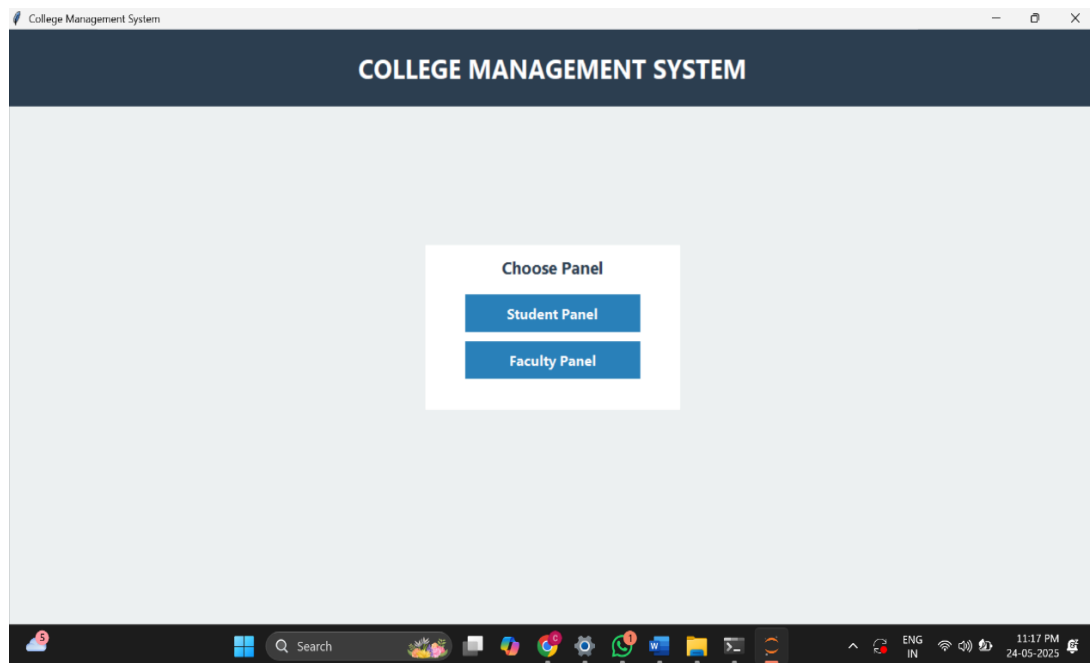


Fig1: Home Page

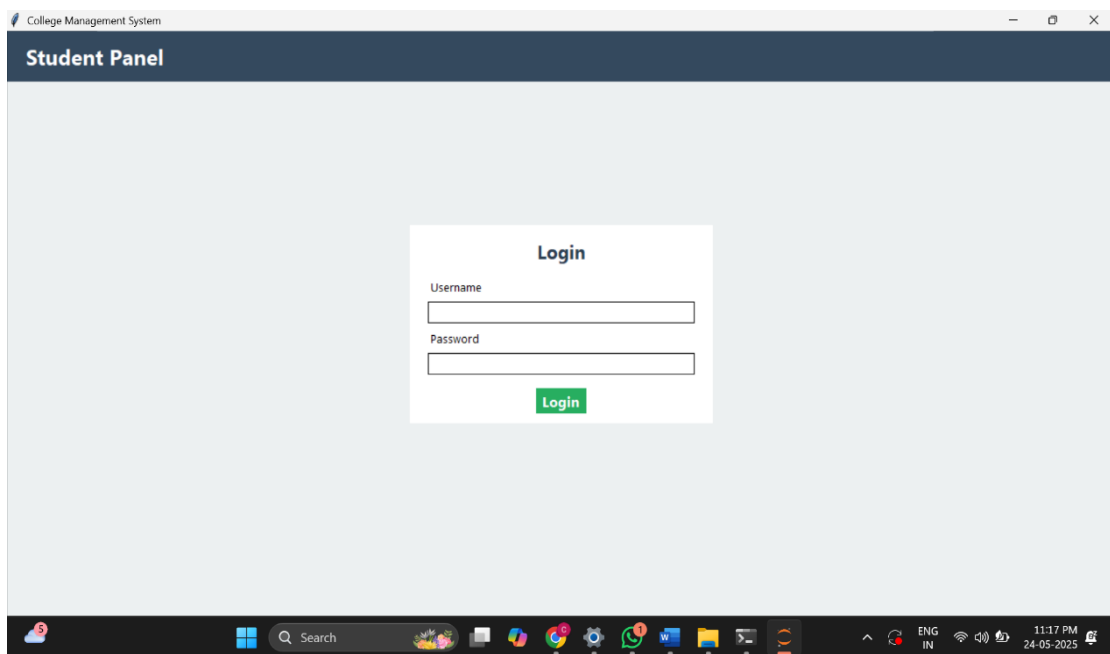


Fig2: Student Login Portal

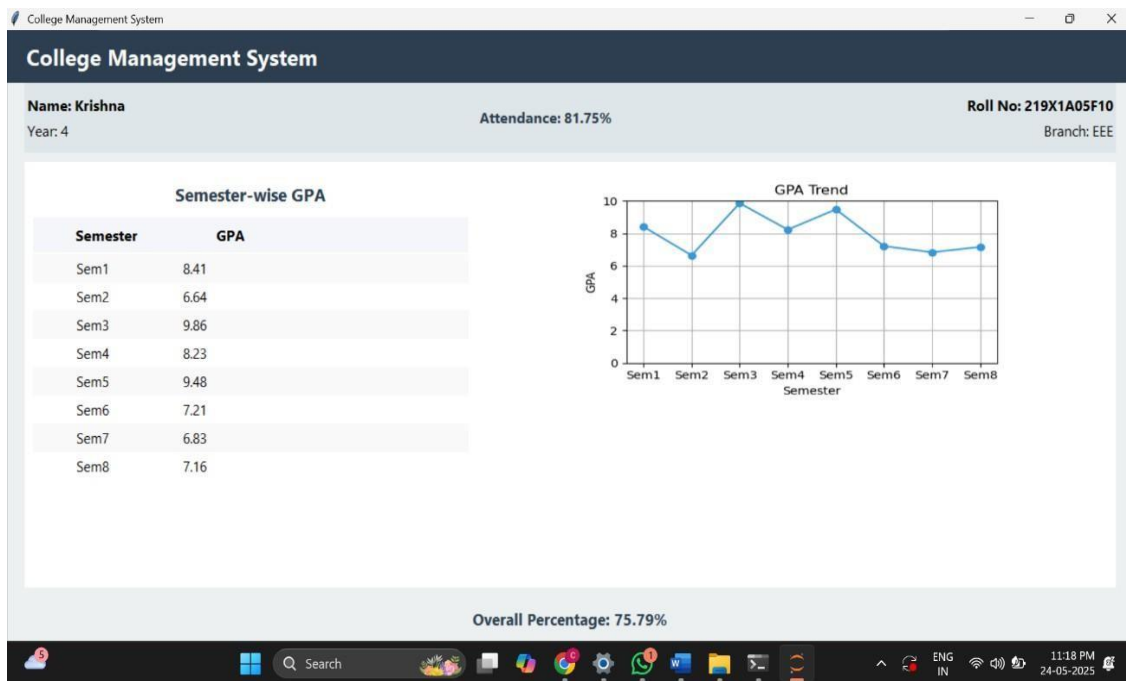


Fig4: Student Details Viewed by Student

The screenshot displays the 'College Management System' interface for a faculty login. The login form is centered on the page and includes fields for Username and Password, along with a Login button.

Login

Username
ML

Password

Login

Fig5: Faculty Login

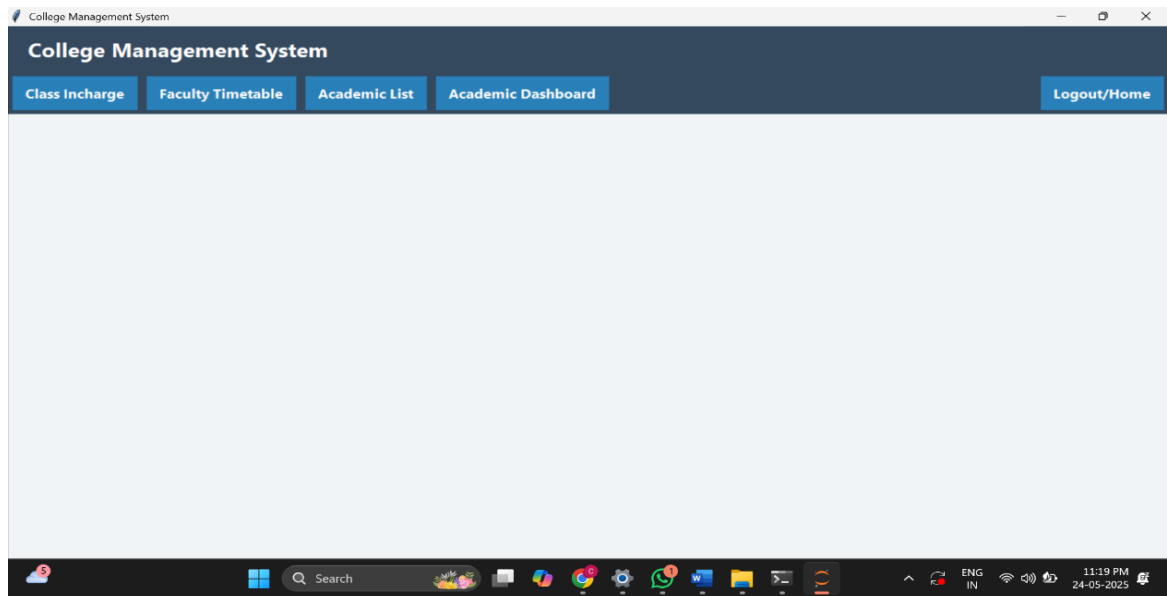


Fig5: Faculty Dashboard

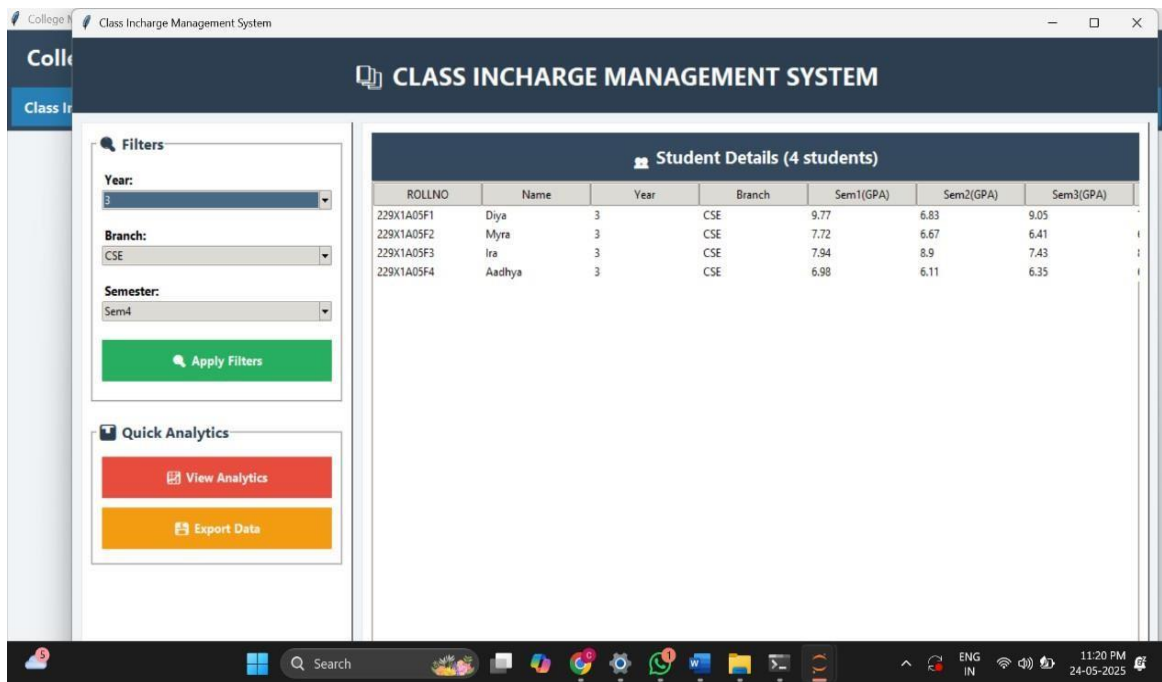


Fig6: Class Incharge Viewing Student Details

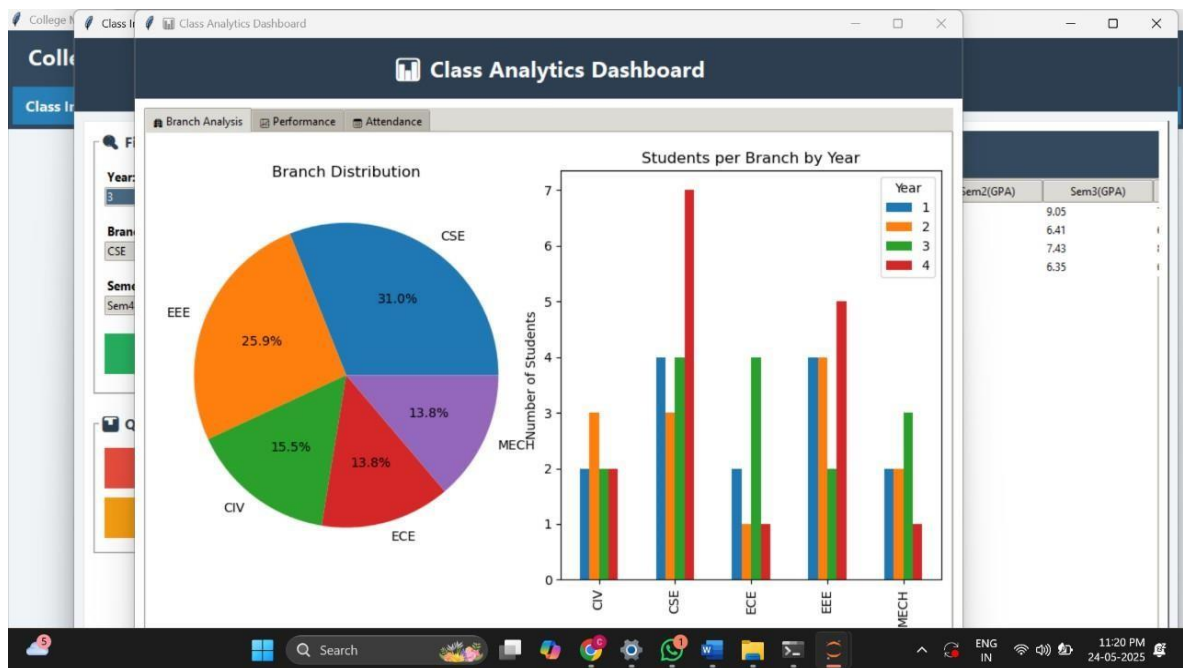


Fig7: Class Analytics

Dr. Suresh Babu's Timetable Subject: ML

Day	Hr-1	Hr-2	Hr-3	Hr-4	Hr-5
Monday	ECE	FREE	CSE	FREE	MECH
Tuesday	FREE	ECE	CSE	FREE	EEE
Wednesday	ECE	MECH	CSE	FREE	EEE
Thursday	FREE	ECE	MECH	CSE	EEE
Friday	CSE	ECE	MECH	FREE	EEE
Saturday	FREE	MECH	ECE	CSE	FREE

Fig8: Class Teacher Timetable

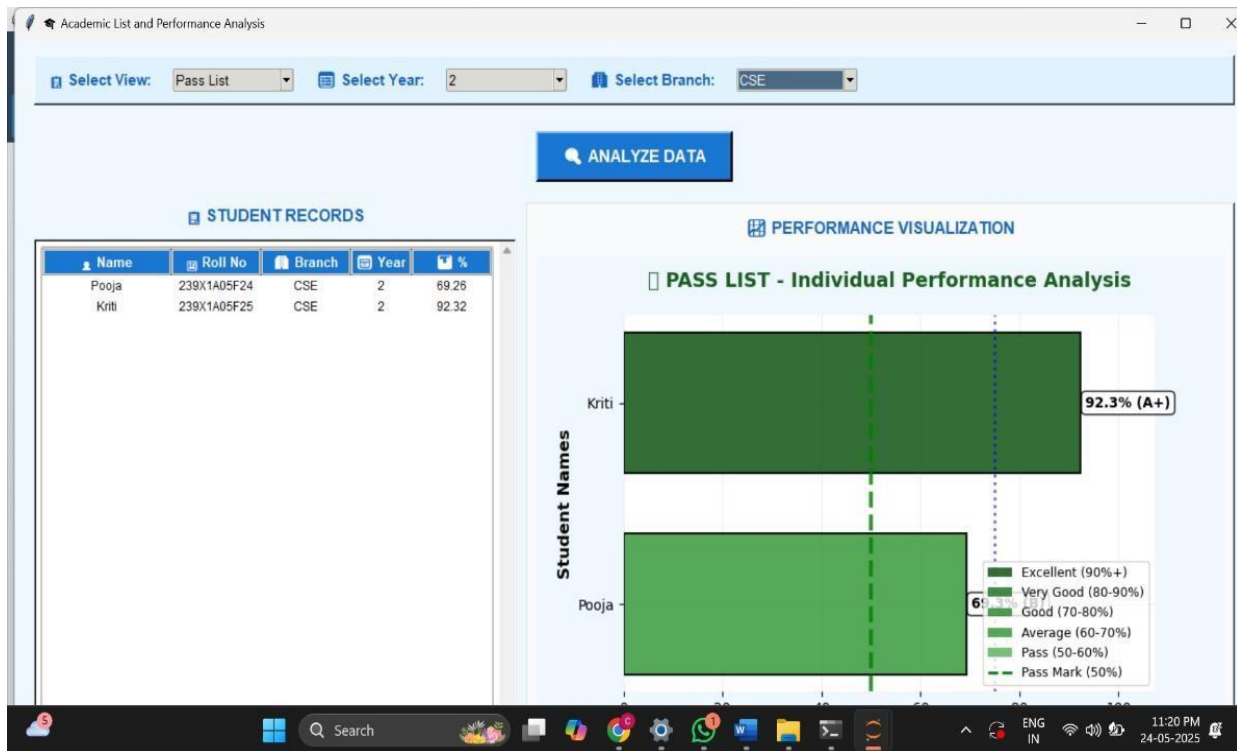


Fig9: Academic List

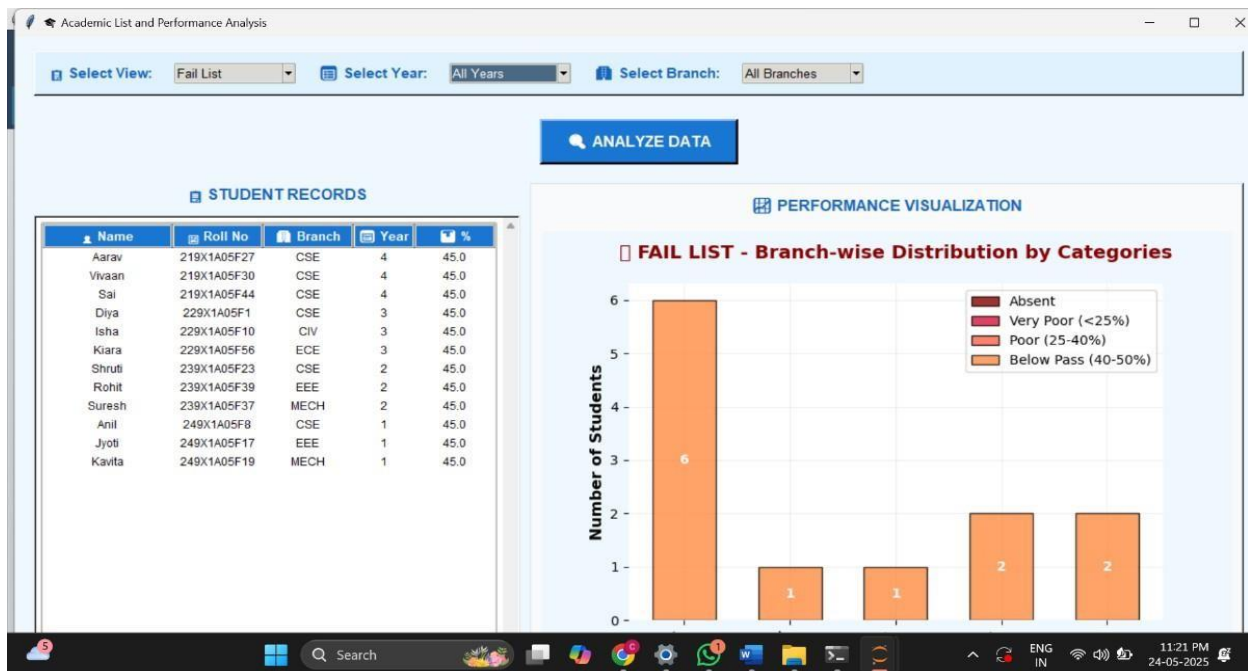


Fig10: Academic List (Fail List)

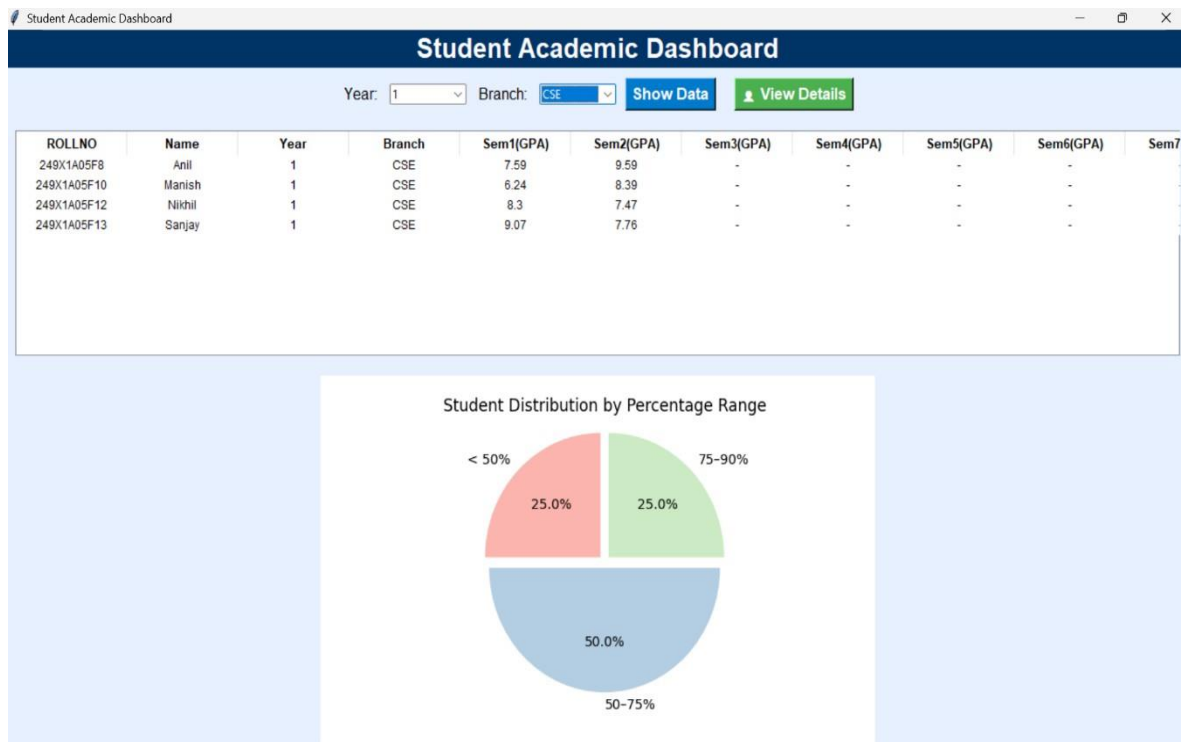


Fig11: Students Dashboard

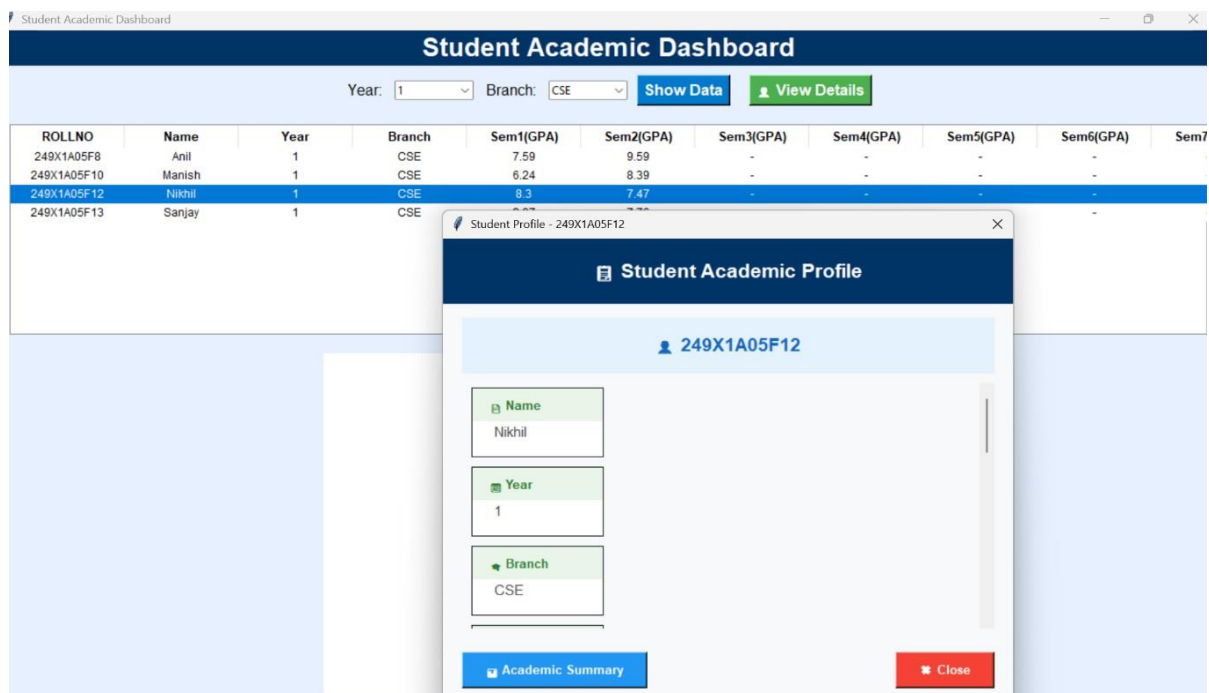


Fig12: Viewing Particular Students Details

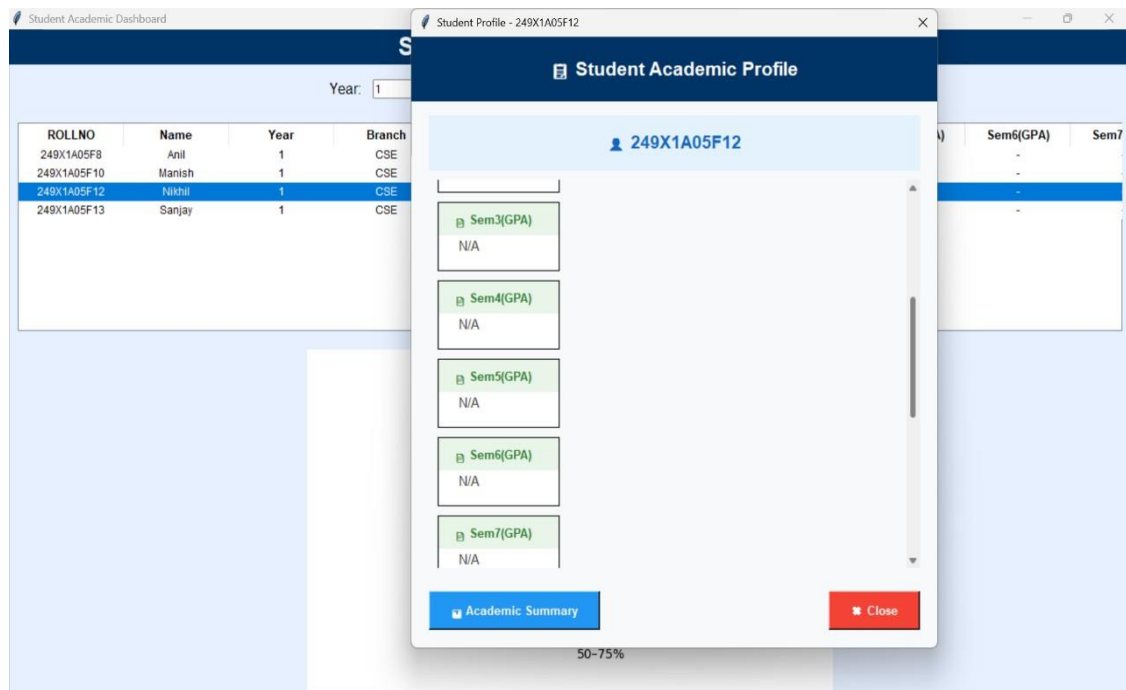


Fig13: Details of a Student

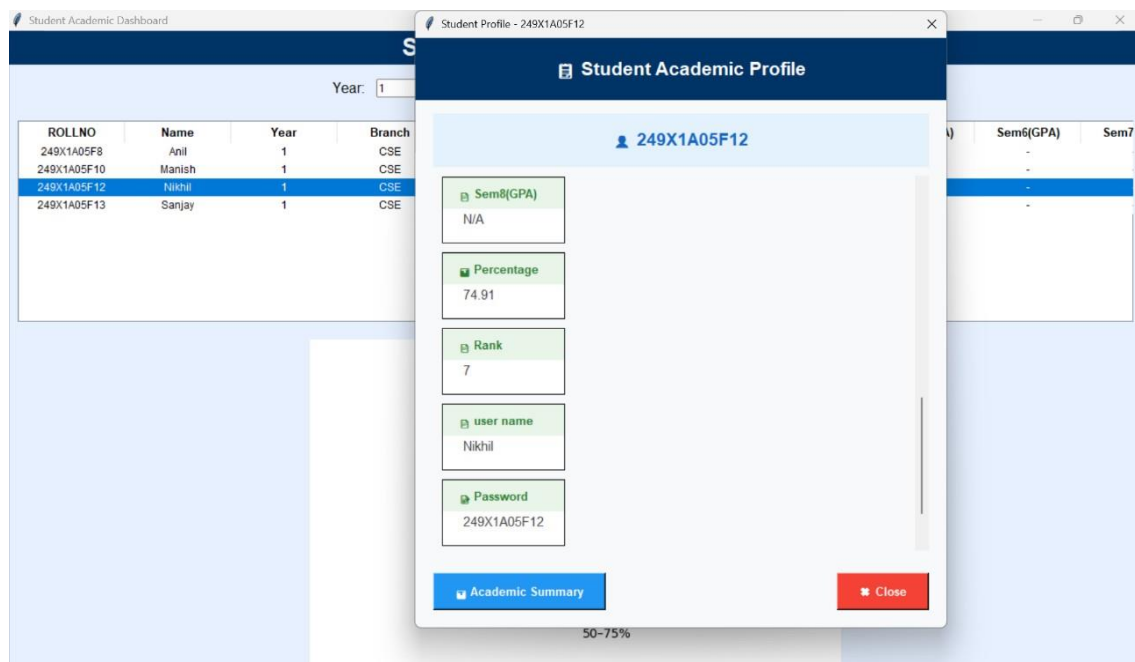


Fig14: Complete Details of Student

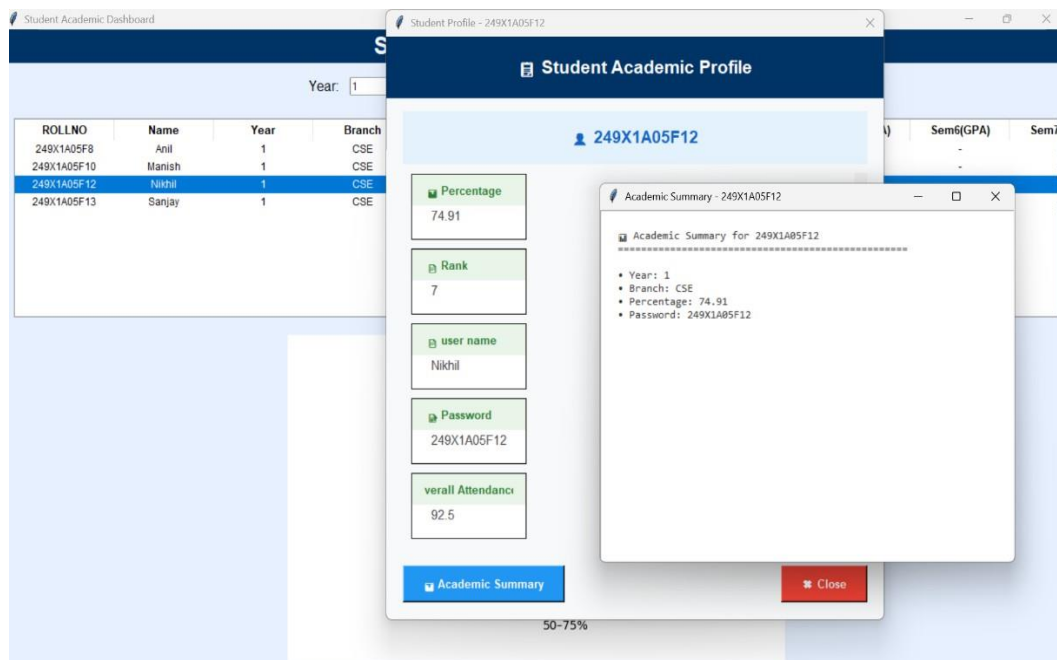


Fig15: Academic Summary of Student

CONCLUSION

The College Management System developed using Python and Tkinter successfully addresses the need for a centralized academic record-keeping platform. It simplifies data entry, reduces redundancy, improves accessibility, and ensures data security through authenticated access. The role-based dashboards offer targeted functionalities for students and faculty, while the use of charts adds analytical capabilities to the academic evaluation process. The application proved efficient and reliable in managing and displaying student performance data. With further enhancements such as database integration and broader platform support, the system has the potential to become a comprehensive academic management solution for schools, colleges, and universities. The project reflects the practical application of programming, GUI design, and data handling to solve real-world institutional problems.

FUTURE WORKS

While the current version of the College Management System provides essential functionalities, there is significant scope for future enhancements. Planned improvements include adding a database (e.g., SQLite or MySQL) for better data scalability and security, integrating email/SMS notifications for grades, and implementing role-based password recovery. Future iterations could also support biometric or OTP-based login for enhanced security. Another proposed addition is a report generation module that allows exporting performance summaries as PDF files. The system can also be extended to include attendance tracking, assignment submission, and parent access portals. An AI-based recommendation system can be incorporated to assist faculty in identifying students needing academic counselling. Additionally, transitioning from a desktop application to a web-based or mobile-friendly version using Django or Flutter could improve accessibility across platforms.

REFERENCES

- Python Software Foundation. Python Language Reference. <https://www.python.org>
- Tkinter Documentation - <https://docs.python.org/3/library/tkinter.html>
- Pandas Library Documentation - <https://pandas.pydata.org/>
- Matplotlib Documentation - <https://matplotlib.org/>
- Real Python Tutorials - <https://realpython.com/>
- Stack Overflow – Community Discussions on GUI, Excel handling, and Chart plotting
- W3Schools – Basic Python and GUI References – <https://www.w3schools.com/python/>

GitHub Repository

Link: <https://github.com/BandiGayathriBhargavi/Student-Report-Generation>